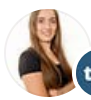


Data preprocessing for Machine Learning in Python

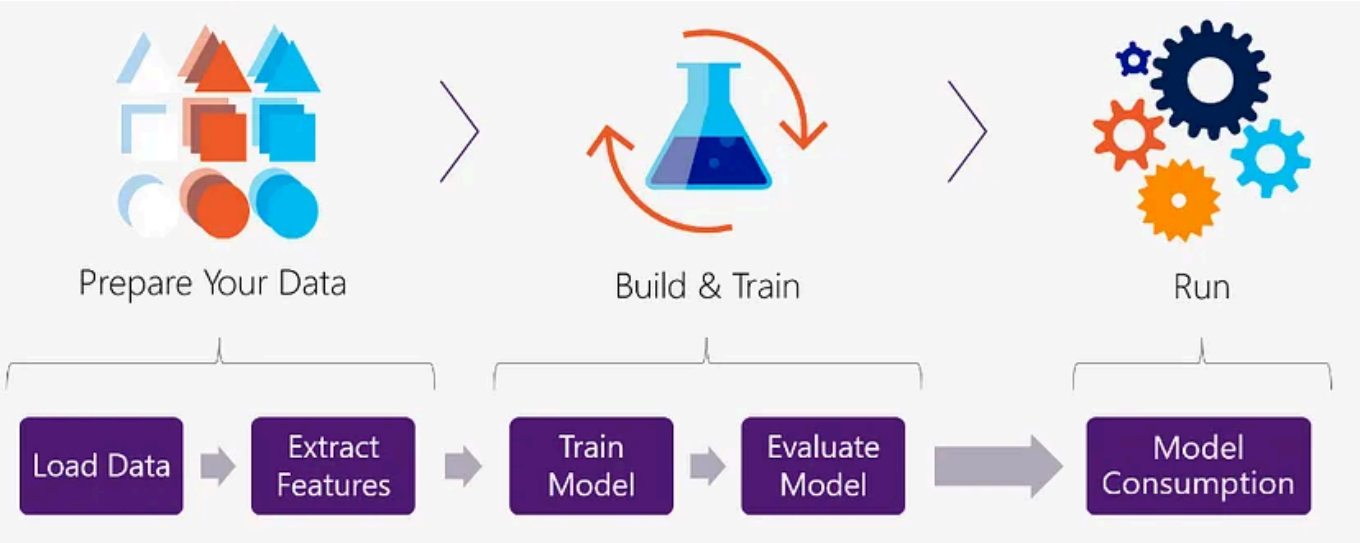


Cvetanka Eftimoska · Follow

Published in Towards Data Science · 8 min read · Apr 30, 2020

 173 



Introduction

Data preprocessing is a crucial step in machine learning and it is very important for the accuracy of the model. Data contains noise, missing values, it is incomplete and sometimes it is in an unusable format which cannot be directly used for machine learning models. But what if we use questionable and dirty data? What will the final result be and can the decision be trusted? Preprocessing data is the key — the goal with the preprocessing is to get more meaningful data that can be trusted. These techniques allow us to transform the raw data into a clean and usable data set and make the data more meaningful by rescaling, standardizing, binarizing and so on.

Data Preprocessing Concepts

In the example below, we will use one file that contains cars and it can be downloaded from <https://www.kaggle.com/antfarol/car-sale-advertisements>. We will use Python for data preprocessing and several libraries of importance:

- Pandas — a package providing fast, flexible and expressive data structures designed to make working with “relational” or “labeled” data

Open in app ↗

Sign upSign in

- Statsmodel — a package that allows users to explore data, estimate statistical models and perform statistical tests;
- Matplotlib — a plotting library;
- Seaborn — a library for making statistical graphics in Python. It is built on top of matplotlib and closely integrated with pandas data structures.

There are a lot of methodologies and concepts that can be used, but in this example, we will only see some of them:

1. Handling missing values
2. Dealing with outliers
3. Multicollinearity
4. Dealing with categorical values
5. Standardization

Load the Data

The first step is loading the data. We can read the .csv file and using the method head we can see the first 5 rows in our data set.

```
raw_data = pd.read_csv('C:\\files\\car_ad.csv')

raw_data.head()
```

	car	price	body	mileage	engV	engType	registration	year	drive
0	Ford	15500.0	crossover	68	2.5	Gas	yes	2010	full
1	Mercedes-Benz	20500.0	sedan	173	1.8	Gas	yes	2011	rear
2	Mercedes-Benz	35000.0	other	135	5.5	Petrol	yes	2008	rear
3	Mercedes-Benz	17800.0	van	162	1.8	Diesel	yes	2012	front
4	Mercedes-Benz	33000.0	vagon	91	NaN	Other	yes	2013	NaN

Discovering the Data

Now, let's discover the data. We can use the describe method — if we use this method we will get only the descriptive statistics of the numerical features. We can include all of them by using include='all'.

```
raw_data.describe(include='all')
```

	car	price	body	mileage	engV	engType	registration	year	drive
count	9576	9309.000000	9576	9576.000000	9142.000000	9576	9576	9576.000000	9065
unique	87	NaN	6	NaN	NaN	4	2	NaN	3
top	Volkswagen	NaN	sedan	NaN	NaN	Petrol	yes	NaN	front
freq	936	NaN	3646	NaN	NaN	4379	9015	NaN	5188
mean	NaN	16081.710894	NaN	138.862364	2.646344	NaN	NaN	2006.605994	NaN
std	NaN	24301.899483	NaN	98.629754	5.927699	NaN	NaN	7.067924	NaN
min	NaN	259.350000	NaN	0.000000	0.100000	NaN	NaN	1953.000000	NaN
25%	NaN	5400.000000	NaN	70.000000	1.600000	NaN	NaN	2004.000000	NaN
50%	NaN	9500.000000	NaN	128.000000	2.000000	NaN	NaN	2008.000000	NaN
75%	NaN	17000.000000	NaN	194.000000	2.500000	NaN	NaN	2012.000000	NaN
max	NaN	547800.000000	NaN	999.000000	99.990000	NaN	NaN	2016.000000	NaN

We can see the unique values and the most common category for every column. For example, we can see that the column registration contains 9015 values of yes and that it has two unique values. Also, we can see the mean, standard deviation, maximum and minimum values and so on.

Handling Missing Values

An easy way to check for missing values is to use the method isnull. We will get a data frame with true (1) and false (0) values, so we will sum the values and we can see in which column we have missing values.

```
data = raw_data.copy()
data.isnull().sum()
```

car	0
price	267
body	0
mileage	0
engV	434
engType	0
registration	0
year	0
drive	511
dtype:	int64

We can handle the missing values in two ways:

1.Eliminate missing values:

a. **Eliminate rows:** we can remove the rows with unacceptable missing values. This can be done by using the method dropna. This way, we will exclude the rows that contain missing values.

```
data_no_missing_values = data.dropna(subset=['engV', 'drive'])
data_no_missing_values.describe(include='all')
```

	car	price	body	mileage	engV	engType	registration	year	drive
count	8739	8495.000000	8739	8739.000000	8739.000000	8739	8739	8739.000000	8739
unique	83	NaN	6	NaN	NaN	4	2	NaN	3
top	Volkswagen	NaN	sedan	NaN	NaN	Petrol	yes	NaN	front
freq	860	NaN	3321	NaN	NaN	4065	8236	NaN	4973
mean	NaN	16185.453305	NaN	140.095434	2.588607	NaN	NaN	2006.609681	NaN
std	NaN	24449.641512	NaN	97.892213	5.416670	NaN	NaN	6.968947	NaN
min	NaN	259.350000	NaN	0.000000	0.100000	NaN	NaN	1959.000000	NaN
25%	NaN	5490.000000	NaN	71.000000	1.600000	NaN	NaN	2004.000000	NaN
50%	NaN	9500.000000	NaN	130.000000	2.000000	NaN	NaN	2008.000000	NaN
75%	NaN	17145.600000	NaN	195.500000	2.500000	NaN	NaN	2012.000000	NaN
max	NaN	547800.000000	NaN	999.000000	99.990000	NaN	NaN	2016.000000	NaN

b. **Eliminate columns:** If the column has 95% or more missing values, it can and needs to be deleted because the estimation of the missing values cannot be trusted as it will be calculated from the 5% (or less) of the data, hence, it is irrelevant.

2. **Estimate missing values:** if only acceptable percentage of values are missing, we can estimate the values. This can be done by filling the values in with the mean, median or most frequent value of the respective feature.

Also, there are multiple methods that can be used for imputing the missing values. We can use Simple imputer, Iterative imputer or Categorical Imputer. Categorical imputer is used for imputing the categorical features.

```
from sklearn.preprocessing import Imputer
imputer = Imputer(missing_values='NaN', strategy='mean')
imputer = imputer.fit(data[['price']])

data_no_missing_values[['price']] = imputer.transform(data_no_missing_values[['price']])
```

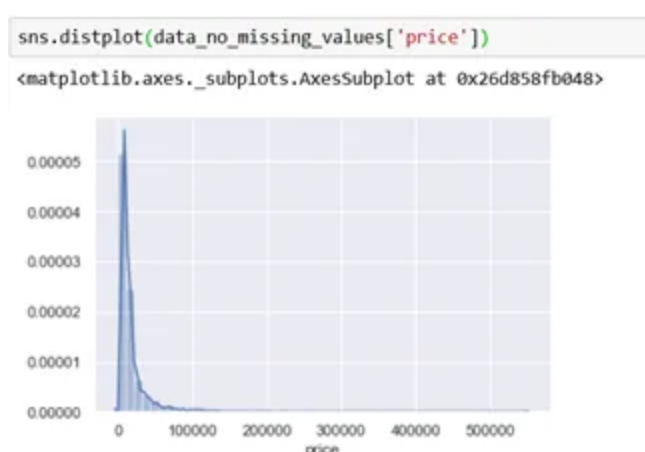
After this, if we call the method **describe** we can see that in the column price we have the same count as car and body. Our missing values are replaced with the mean of all values in this column.

```
data_no_missing_values.describe(include='all')
```

	car	price	body	mileage	engV	engType	registration	year	drive
count	8739	8739.000000	8739	8739.000000	8739.000000	8739	8739	8739.000000	8739
unique	83	NaN	6	NaN	NaN	4	2	NaN	3
top	Volkswagen	NaN	sedan	NaN	NaN	Petrol	yes	NaN	front
freq	860	NaN	3321	NaN	NaN	4065	8236	NaN	4973
mean	NaN	16182.556732	NaN	140.095434	2.588607	NaN	NaN	2006.609681	NaN
std	NaN	24105.864611	NaN	97.892213	5.416670	NaN	NaN	6.968947	NaN
min	NaN	259.350000	NaN	0.000000	0.100000	NaN	NaN	1959.000000	NaN
25%	NaN	5500.000000	NaN	71.000000	1.600000	NaN	NaN	2004.000000	NaN
50%	NaN	9800.000000	NaN	130.000000	2.000000	NaN	NaN	2008.000000	NaN
75%	NaN	16800.000000	NaN	195.500000	2.500000	NaN	NaN	2012.000000	NaN
max	NaN	547800.000000	NaN	999.000000	99.990000	NaN	NaN	2016.000000	NaN

Dealing with Outliers

If we are working with regression, this is a very important topic. We will plot the distribution for the column price. For an optimal result, we are looking for a normal distribution. But, the price has an exponential one. This will be a problem if we are using regression. We can see in the table above, that the minimum value for the price is 259, the maximum is 547,800 and the mean value is 16,182. Also, we can see that 25% of the values are under 5,500 and 75% are under 16,800. So, in this case, we have outliers. Outliers are observations that lie on abnormal distance from other observations in the data and they will affect the regression dramatically. Because of this, the regression will try to place the line closer to these values.



The common rule is to remove everything that is 3 * standard deviations far from the mean. What does this mean? For normally distributed values there is a known rule: 68–95–99.7. The rule can be explained as:

- 68% of the data is distributed in the interval [mean — standard deviation, mean + standard deviation],
- 95% of the data is distributed in the interval [mean — 2 * standard deviation, mean + 2 * standard deviation],
- 99.7% of the data is distributed in the interval [mean — 3 * standard deviation, mean + 3 * standard deviation].

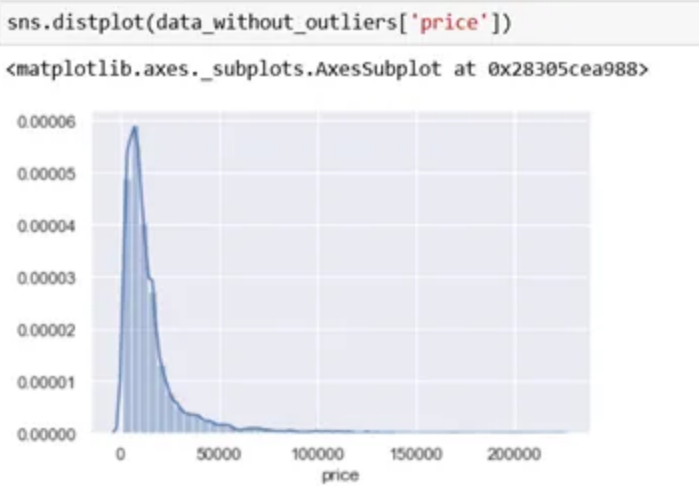
Based on this, we can say that the values that are out of the interval [mean — 3*std, mean + 3*std] are outliers and these values can be removed.

```
q = data_no_missing_values['price'].quantile(0.99865)
q1 = data_no_missing_values['price'].quantile(0.00135)
data_without_outliers_t = data_no_missing_values[data_no_missing_values['price']<q]
data_without_outliers = data_without_outliers_t[data_without_outliers_t['price']>q1]

data_without_outliers.describe(include='all')
```

	car	price	body	mileage	engV	engType	registration	year	drive
count	8715	8715.000000	8715	8715.000000	8715.000000	8715	8715	8715.000000	8715
unique	83	NaN	6	NaN	NaN	4	2	NaN	3
top	Volkswagen	NaN	sedan	NaN	NaN	Petrol	yes	NaN	front
freq	960	NaN	3305	NaN	NaN	4043	8214	NaN	4971
mean	NaN	15749.876415	NaN	140.320367	2.584554	NaN	NaN	2006.631670	NaN
std	NaN	20367.751122	NaN	97.806313	5.422062	NaN	NaN	6.900866	NaN
min	NaN	550.000000	NaN	0.000000	0.100000	NaN	NaN	1961.000000	NaN
25%	NaN	5500.000000	NaN	72.000000	1.600000	NaN	NaN	2004.000000	NaN
50%	NaN	9800.000000	NaN	130.000000	2.000000	NaN	NaN	2008.000000	NaN
75%	NaN	16800.000000	NaN	196.000000	2.500000	NaN	NaN	2012.000000	NaN
max	NaN	222000.000000	NaN	999.000000	99.990000	NaN	NaN	2016.000000	NaN

The maximum value is still far away from the mean, but it is acceptably closer. We can now plot the distribution and we can see that the data is still distributed the same way, but with fewer outliers. We can do this with the column mileage as well.



Multicollinearity

Multicollinearity exists whenever an independent variable is highly correlated with one or more of the other independent variables in a multiple regression equation. Multicollinearity is a problem because it undermines the statistical significance of an independent variable.

Unfortunately, using scikit-learn we cannot straight forward determine if multicollinearity exists. We can do this by using statsmodels. One of the best ways to check for multicollinearity is through VIF (variance inflation factor). VIF produces a measure that estimates the degree to which the square root of the standard error of an estimate is bigger, compared to a situation when the variable was completely uncorrelated with the other predictors. The numerical value for VIF tells us (in decimal form) the percentage the

variance (i.e. the standard error squared) is inflated for each coefficient. For example, a VIF of 1.9 tells us that the variance of a particular coefficient is 90% bigger than what we would expect if there was no multicollinearity — if there was no correlation with other predictors.

In our example, we can say: if mileage is smaller — the price will be bigger. Also, if the car is older, the price will be smaller. So, we can check for multicollinearity for these three columns.

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant
variables = data_without_outliers[['price','mileage', 'year']]
vif = pd.DataFrame()
df = add_constant(variables)
vif["VIF"] = [variance_inflation_factor(df.values, i) for i in range(df.shape[1])]
vif["features"]=df.columns

vif
```

	VIF	features
0	127630.218781	const
1	1.270862	price
2	1.410170	mileage
3	1.505335	year

When VIF value is equal to 1, there is no multicollinearity at all. Values between 1 and 5 are considered perfectly okay. But, there is no precise definition of what values would be acceptable. We can find in different resources that values below 5, 6 or even 10 are acceptable. So, in our example, we can see that all of the values are below 5 and they are okay. We can rarely find data where all the features have value below 5. We can say that our data set doesn't have multicollinearity. If we had multicollinearity we should drop the column using the method drop. As input we will set the column that we want to be removed and axis = 1 — this means that we will delete a column. For example, let's say that we want to delete the column year from our dataset.

```
data_without_outliers = data_without_outliers.drop(['year'], axis=1)
```

Dealing with Categorical Values

Dummy coding is a commonly used method for converting a categorical input variable into a continuous variable. ‘Dummy’, as the name suggests, is a duplicate variable that represents one level of a categorical variable. The presence of a level is represented by 1 and absence is represented by 0. For every level present, one dummy variable will be created. In Python, we can

use the method `get_dummies` which returns a new data frame that contains one-hot-encoded columns.

```
data_dummies = pd.get_dummies(data_without_outliers)
data_dummies
```

	price	mileage	engV	year	car_Acura	car_Alfa Romeo	car_Aro	car_Aston Martin	car_Audi	car_BMW	...	body_van	engType_Diesel	engType_Gas
0	15500.0	68	2.5	2010	0	0	0	0	0	0	...	0	0	1
1	20500.0	173	1.8	2011	0	0	0	0	0	0	...	0	0	1
2	35000.0	135	5.5	2008	0	0	0	0	0	0	...	0	0	0
3	17800.0	162	1.8	2012	0	0	0	0	0	0	...	1	1	0
5	16600.0	83	2.0	2013	0	0	0	0	0	0	...	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
9571	14500.0	140	2.0	2011	0	0	0	0	0	0	...	0	0	1
9572	2200.0	150	1.6	1986	0	0	0	0	0	0	...	0	0	0
9573	18500.0	180	3.5	2008	0	0	0	0	0	0	...	0	0	0
9574	16999.0	150	3.5	2008	0	0	0	0	0	0	...	0	0	1
9575	22500.0	71	3.6	2007	0	0	0	0	1	0	...	0	0	0

Standardization

After dealing with categorical values, we can standardize the values. They are not on the same scale; therefore, our model will give more weightage to the ones that have bigger values which is not the ideal scenario as the other columns are important for building the model. In order to avoid this issue, we can perform standardization.

Note: In some examples, the standardization might not give better results — they can be even worse.

We will use the class `StandardScaler` from `scikit-learn`. The idea behind `StandardScaler` is that it will transform the data such that its distribution will have a mean value 0 and a standard deviation of 1.

```
from sklearn.preprocessing import StandardScaler
std = StandardScaler()
data_scaled = data_dummies.copy()
data_scaled = data_dummies
data_scaled = std.fit_transform(data_dummies)
data_scaled = std.transform(data_dummies)

data_scaled

array([[ -0.91924398, -1.46108012, -0.47441504, ..., -3.5148891 ,
         3.04454985, -1.60193465],
       [ -0.91922402, -1.44997338, -0.49807887, ..., -3.5148891 ,
        -1.98961117,  6.16572504],
       [ -0.91916615, -1.45399296, -0.37299864, ..., -3.5148891 ,
        -1.98961117,  6.16572504],
       ...,
       [ -0.919232  , -1.44923293, -0.44060957, ..., -3.5148891 ,
         3.04454985, -1.60193465],
       [ -0.91923799, -1.45240629, -0.44060957, ...,  0.57679824,
        -1.98961117, -1.60193465],
       [ -0.91921604, -1.46076278, -0.43722902, ..., -3.5148891 ,
         3.04454985, -1.60193465]])
```

Conclusion

The aim of this article is to show how some of the data preprocessing techniques can be used and gain a deeper understanding of the situations

where those techniques can be applied. There are other forms of data cleaning that are useful as well, but for now, we are covering those that are important before the formulation of any model. Better and cleaner data outperforms the best algorithms. If we use a very simple algorithm on data that is cleaned, we will get very impressive and accurate results.

If you are interested in this topic, do not hesitate to contact me.

LinkedIn profile: <https://www.linkedin.com/in/ceftimoska/>

The original blogpost is available on the following link:
<https://interworks.com.mk/data-preprocessing-for-machine-learning-in-python/>

Also, you can find another similar blogpost on the following link:
<https://interworks.com.mk/focusareas/data-management/>

- Machine Learning
- Data Science
- Data Preprocessing



Published in Towards Data Science

800K Followers · Last published 4 hours ago

Follow



Written by Cvetanka Eftimoska

54 Followers · 67 Following

Follow

Data science enthusiast with knowledge in Machine learning, Python, R, Statistics, Big Data and Apache Spark. Seeking to increase data efficiency.

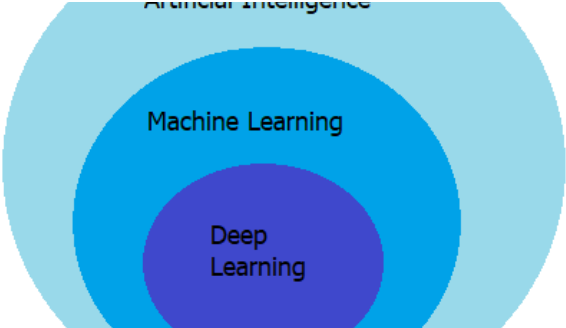
No responses yet



What are your thoughts?

Respond

More from Cvetanka Eftimoska and Towards Data Science



tds

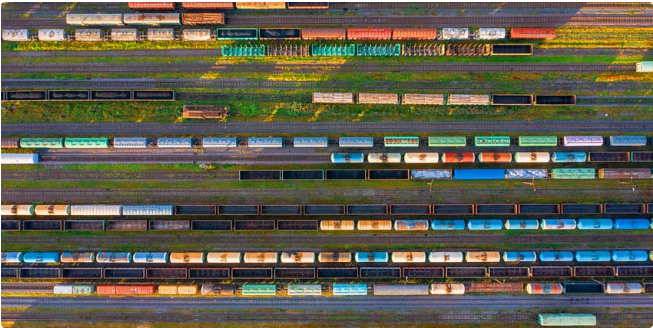
In Towards Data Science by Cvetanka Eftimoska

Deep Learning with TensorFlow

In this article, you can read how we can use tensor flow for deep learning in Python

May 26, 2020

148



tds

In Towards Data Science by Dr. Janna Lipenkova

Injecting domain expertise into your AI system

How to connect the dots between AI technology and real life

2d ago

226

6



tds

In Towards Data Science by Peng Qian

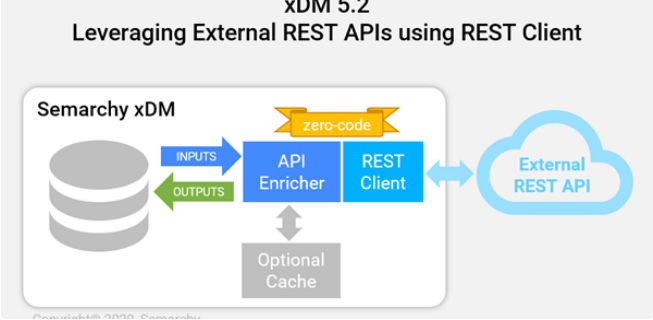
Using LLamaIndex Workflow to Implement an Agent Handoff...

Example: a customer service chatbot project

2d ago

103

2



Cvetanka Eftimoska

Using Semarchy to Master Data Management

What does master data mean? What is master data management?

Dec 19, 2023

2

Recommended from Medium



 In Stackademic by Khouloud Haddad Amamou

Data Analysis with Python Pandas and Matplotlib (Advanced)

1. Introduction

4d ago  91



 In Pipeline: Your Data Engineering R... by Zach Qu...

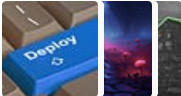
3 Data Science Projects That Got Me 12 Interviews. And 1 That Got...

3 work samples that got my foot in the door, and 1 that almost got me tossed out.

 Aug 30, 2022  5.3K  86



Lists



Predictive Modeling w/ Python

20 stories · 1812 saves



Practical Guides to Machine Learning

10 stories · 2185 saves



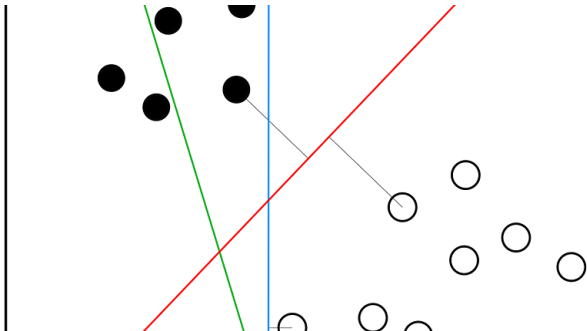
Natural Language Processing

1912 stories · 1569 saves



data science and AI

40 stories · 324 saves



 In Artificial Intelligence in Plain En... by Ritesh Gu...

Data Science All Algorithm Cheatsheet 2025

Stories, strategies, and secrets to choosing the perfect algorithm.



 In Level Up Coding by Youssef Hosni

13 SQL Statements for 90% of Your Data Science Tasks

Structured Query Language (SQL) is a programming language designed for...

★

Jan 6

👤 815

💬 16

🔖

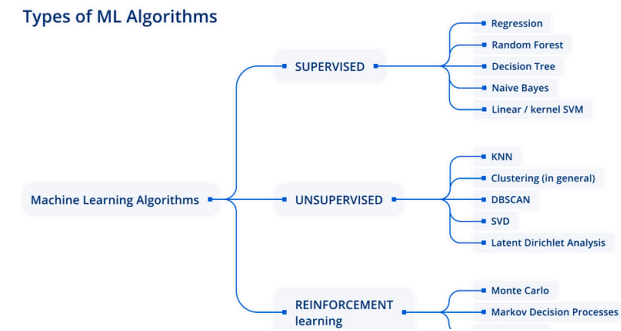
★

Feb 27, 2023

👤 4.3K

💬 57

🔖



 John Vastola

10 Must-Know Machine Learning Algorithms for Data Scientists

Machine learning is the science of getting computers to act without being explicitly...

★

Dec 6, 2022

👤 1.3K

💬 22

🔖



tds

 In Towards Data Science by Matt Chapman

The Portfolio that Got Me a Data Scientist Job

Spoiler alert: It was surprisingly easy (and free) to make

★

Mar 25, 2023

👤 5.5K

💬 95

🔖

See more recommendations